

Lab 15 - Secrets, ConfigMaps and Safe Injection

Field	Value
Lab ID	Lab 15
Day / Topic	Day 3 Security
Duration	50 minutes
Namespace	kcna-lab15
Mapping	LO4 A4 K6
Cluster	Single-node kind cluster, Kubernetes v1.30 or later

Safety notice

Every credential-shaped value in this lab is an obvious placeholder such as

DUMMY-not-a-real-password-change-me. Nothing here authenticates to anything.

Do not substitute a real credential at any point, including in data/.

1. Objective

By the end of this lab you will be able to:

1. Create ConfigMaps three ways - `--from-literal`, `--from-file` and `--from-env-file` - and predict the resulting keys.
2. Inject configuration as environment variables and as volumes, and choose between them for a given requirement.
3. Demonstrate, with your own hands, that `**a Secret is base64-encoded, not encrypted**`, and name the three mitigations that actually protect it.
4. Explain immutable: true and the version-the-name pattern it forces.
5. Explain why a subPath mount never receives updates while a whole-directory mount does.
6. Diagnose `CreateContainerConfigError` from a key-name mismatch, using the real kubelet event text.

2. Prerequisites

- Lab 14 completed - you need to understand `*who*` can read a Secret before you learn how one is stored.
- A running single-node kind cluster.
- Working directory:

```
cd courseware/labs/lab-15-secrets-configmaps
pwd
```

Expected output:

```
/.../courseware/labs/lab-15-secrets-configmaps
```

- A base64 binary that can decode. Check which flag yours uses:

```
printf 'hello' | base64 | base64 -d
```

Expected output on Linux and on macOS (both accept -d; older macOS releases need -D):

```
hello
```

3. Scenario

Meridian Freight's booking-api reads a JSON settings file, a set of feature flags, and a database credential. On the legacy VMs all three lived in `/opt/booking/.env`, world-readable, in the same Git repository as the code.

Your task is to split them correctly: non-secret configuration into ConfigMaps, credentials into a Secret, injected in the safest form for each. You will then show the security team exactly how much protection a Kubernetes Secret does and does not provide - because they have been told "Kubernetes encrypts secrets", and that is not true by default.

4. Step-by-step procedure

Step 1 - Namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab15 created
```

Step 2 - Look at the source data

```
cat data/app-settings.json
```

Expected output:

```
{
  "service": "booking-api",
  "environment": "training",
  "region": "ap-southeast-1",
  "listen_port": 8080,
  "log_level": "info",
  "quote_cache_ttl_seconds": 300,
  "downstream": {
    "quote_api": "http://quote-api.kcna-lab11.svc.cluster.local:80",
    "tracking_api": "http://tracking-api.kcna-lab15.svc.cluster.local:80"
  },
  "limits": {
    "max_containers_per_booking": 40,
    "max_weight_kg": 26000
  }
}
```

```
cat data/feature-flags.env
```

Expected output:

```
# Consumed by: kubectl create configmap ... --from-env-file=data/feature-flags.env
# Each line becomes ONE ConfigMap key. Blank lines and # comments are ignored.
FEATURE_MULTILEG_QUOTES=true
FEATURE_LIVE_VESSEL_ETA=false
FEATURE_BULK_UPLOAD=true
FEATURE_LEGACY_EDI=false
```

Step 3 - Three ways to build a ConfigMap

Generate - do not yet apply - the ConfigMap, so you can see how each flag maps:

```
kubectl create configmap booking-app-config -n kcna-lab15 \
  --from-file=app-settings.json=data/app-settings.json \
  --from-env-file=data/feature-flags.env \
  --from-literal=APP_GREETING="Meridian Freight booking API" \
  --dry-run=client -o yaml | grep -vE '^s{4}' | head -20
```

Expected output (the grep -v hides the JSON body so the key list is readable):

```
apiVersion: v1
data:
  APP_GREETING: Meridian Freight booking API
  FEATURE_BULK_UPLOAD: "false"
  app-settings.json: |
kind: ConfigMap
metadata:
  creationTimestamp: null
  name: booking-app-config
  namespace: kcna-lab15
```

The abridged view above only proves the *shape*. Run the command without the
grep/head filters to see all seven keys.

The mapping rule for each flag:

Flag	Key becomes	Value becomes
--from-file=app-settings.json=data/app-settings.json	the name left of = (app-settings.json); without key=, the basename of the file	the entire file contents
--from-env-file=data/feature-flags.env	one key per line, the part before =	the part after =. # comments and blank lines are ignored
--from-literal=APP_GREETING=...	exactly what you typed	exactly what you typed

Now apply the checked-in declarative version, which is the same object:

```
kubectl apply -f manifests/10-configmap-booking-app-config.yaml
```

Expected output:

```
configmap/booking-app-config created
```

```
kubectl get configmap booking-app-config -n kcna-lab15 -o jsonpath='{range .data}{"{""}{end}" 2>/dev/null
kubectl describe configmap booking-app-config -n kcna-lab15 | head -22
```

Expected output:

```
Name:          booking-app-config
Namespace:     kcna-lab15
Labels:        app=booking-api
Annotations:    <none>

Data
====
APP_GREETING:
----
Meridian Freight booking API

FEATURE_BULK_UPLOAD:
----
true

FEATURE_LEGACY_EDI:
----
false

FEATURE_LIVE_VESSEL_ETA:
----
false
```

Look at what describe just did. It printed every ConfigMap value in full,

in plaintext, to your terminal. Hold that thought until Step 6.

Confirm the ConfigMap value is byte-identical to the file it came from:

```
diff <(kubectl get configmap booking-app-config -n kcna-lab15 -o jsonpath='{.data.app-settings\.json}') data/app-settings.json && echo "IDENTICAL"
```

Expected output:

```
IDENTICAL
```

Step 4 - Immutability

```
kubectl apply -f manifests/15-configmap-immutable.yaml
```

Expected output:

```
configmap/booking-app-config-v1 created
```

```
kubectl get configmap booking-app-config-v1 -n kcna-lab15 -o jsonpath='{.immutable}{"\n"}'
```

Expected output:

```
true
```

Try to change it:

```
kubectl patch configmap booking-app-config-v1 -n kcna-lab15 \
  --type=merge -p '{"data":{"RELEASE_CHANNEL":"beta"}}'
```

Expected output (rejected):

```
Error from server: failed to patch: ConfigMap "booking-app-config-v1" is invalid: data: Field is immutable
```

Why you would want this:

- A running Pod cannot be broken by someone editing shared config.
- kubelet stops watching the object, which measurably reduces API server and etcd

load on large clusters.

What it costs: the only way to change the content is to delete and recreate. So you version the name (...-v1, ...-v2) and roll the Deployment to point at the new one - which also gives you a real rollout and a real rollback, instead of a silent in-place mutation.

Note that immutable itself can be set on an existing ConfigMap, but never unset:

```
kubectl patch configmap booking-app-config-v1 -n kcna-lab15 \
  --type=merge -p '{"immutable":false}'
```

Expected output:

```
Error from server: failed to patch: ConfigMap "booking-app-config-v1" is invalid: immutable: Field is immutable
```

Step 5 - Create the Secret from data/

Look at the fixture first. Note how loudly it announces that it is fake:

```
cat data/dummy-credentials.env | tail -3
```

Expected output:

```
BOOKING_DB_USERNAME=DUMMY-not-a-real-username
BOOKING_DB_PASSWORD=DUMMY-not-a-real-password-change-me
BOOKING_API_TOKEN=DUMMY-not-a-real-token-0000000000
```

Create the Secret imperatively:

```
kubectl create secret generic booking-credentials -n kcna-lab15 \
  --from-env-file=data/dummy-credentials.env
```

Expected output:

```
secret/booking-credentials created
```

****Notice what is *not* in manifests/.**** There is no booking-credentials YAML

file in this repository, on purpose: ****a Secret manifest must never be committed to a Git repository****, because a base64 string is not a protection (Step 6 proves it).

The only Secret manifest checked in - 20-secret-declarative-example.yaml - exists

to teach the object's *shape* and contains nothing but placeholders. Apply it now so

you can compare stringData with data:

```
kubectl apply -f manifests/20-secret-declarative-example.yaml
```

Expected output:

```
secret/booking-api-example created
```

```
kubectl get secrets -n kcna-lab15
```

Expected output:

NAME	TYPE	DATA	AGE
booking-api-example	Opaque	2	4s
booking-credentials	Opaque	3	30s

Step 6 - Prove that a Secret is encoded, not encrypted

This is the most important five minutes of the lab.

```
kubectl get secret booking-credentials -n kcna-lab15 -o yaml
```

Expected output:

```

apiVersion: v1
data:
  BOOKING_API_TOKEN: RFVNTVktbm90LWEtcVhbc10b2t1bi0wMDAwMDAwMDAw
  BOOKING_DB_PASSWORD: RFVNTVktbm90LWEtcVhbc1wYXNzd29yZC1jaGFuZ2UtbWU=
  BOOKING_DB_USERNAME: RFVNTVktbm90LWEtcVhbc1c2VybmFtZQ==
kind: Secret
metadata:
  creationTimestamp: "2026-09-05T10:31:07Z"
  name: booking-credentials
  namespace: kcna-lab15
  resourceVersion: "4821"
  uid: 6a0f4a1e-6d64-4f9e-b2ab-27c0e0b1f0c2
type: Opaque

```

That looks reassuringly unreadable. It is not. Decode it:

```

kubectl get secret booking-credentials -n kcna-lab15 \
  -o jsonpath='{.data.BOOKING_DB_PASSWORD}' | base64 -d ; echo

```

Expected output:

```
DUMMY-not-a-real-password-change-me
```

Decode every key at once:

```

kubectl get secret booking-credentials -n kcna-lab15 -o json \
  | grep -E '"BOOKING_' \
  | sed 's/["],[/g' \
  | while read -r k v; do printf '%s %s\n' "$k" "$(${printf '%s' "$v" | base64 -d})"; done

```

Expected output:

```

BOOKING_API_TOKEN: DUMMY-not-a-real-token-000000000000
BOOKING_DB_PASSWORD: DUMMY-not-a-real-password-change-me
BOOKING_DB_USERNAME: DUMMY-not-a-real-username

```

And confirm that stringData is not stored as you typed it:

```

kubectl get secret booking-api-example -n kcna-lab15 -o jsonpath='{.data.EXAMPLE_PASSWORD}' ;
echo
kubectl get secret booking-api-example -n kcna-lab15 -o jsonpath='{.stringData}' ; echo "<-
stringData is write-only"

```

Expected output:

```

RFVNTVktbm90LWEtcVhbc1wYXNzd29yZC1jaGFuZ2UtbWU=
<- stringData is write-only

```

stringData is a write-only convenience field. The API server base64-encodes it into data and it never comes back on a read.

State the conclusion precisely

Base64 is an encoding, not a cipher. It has no key. Anyone - or any process -

that can get the Secret object has the plaintext. By default the value is also

written to etcd in plaintext.

Contrast this with the *one* thing Kubernetes does do for you:

```
kubectl describe secret booking-credentials -n kcna-lab15
```

Expected output:

```
Name:          booking-credentials
Namespace:     kcna-lab15
Labels:        <none>
Annotations:   <none>

Type: Opaque

Data
====
BOOKING_API_TOKEN:    33 bytes
BOOKING_DB_PASSWORD: 35 bytes
BOOKING_DB_USERNAME: 25 bytes
```

describe masks the values (it shows byte counts) - unlike describe configmap in Step 3, which printed everything. That is a courtesy against shoulder-surfing, not a security control: `kubectl get -o yaml` bypasses it entirely.

The mitigations that actually work

Mitigation	What it addresses	How
RBAC on secrets	Who can read the object at all	Lab 14. Note that the built-in view ClusterRole deliberately excludes Secrets. Grant get secrets with resourceNames where you can
Encryption at rest	The plaintext sitting in etcd	--encryption-provider-config on kube-apiserver, with an aescbc/aesgcm provider or, better, a kms v2 provider backed by an external KMS
An external secret store	Removing the secret from Kubernetes entirely	HashiCorp Vault, cloud secret managers, the External Secrets Operator, or the Secrets Store CSI Driver - the material is fetched at mount time and never persisted in etcd
automountServiceAccountToken: false	Stopping a compromised Pod from reading <i>other</i> Secrets via the API	Lab 14, Step 6
Prefer volumes over env	Leakage through /proc/<pid>/environ, child processes and crash dumps	Step 8 below

Check whether this cluster encrypts at rest - the honest answer on kind:

```
kubectl get pod -n kube-system -l component=kube-apiserver \
-o jsonpath='{.items[0].spec.containers[0].command}' \
| tr ',' '\n' | grep -i 'encryption-provider' \
|| echo "NOT SET -> Secrets are stored in etcd in plaintext on this cluster"
```

Expected output on a stock kind cluster:

```
NOT SET -> Secrets are stored in etcd in plaintext on this cluster
```

That is the true state of the training cluster, and of a great many production clusters. Say so to your security team rather than repeating the myth.

Step 7 - Inject as environment variables

```
kubectl apply -f manifests/30-pod-env-injection.yaml
kubectl wait --for=condition=Ready pod/env-consumer -n kcna-lab15 --timeout=90s
```

Expected output:

```
pod/env-consumer created
pod/env-consumer condition met
```

```
kubectl exec -n kcna-lab15 env-consumer -- sh -c 'env | grep -E "^(
GREETING|MULTILEG_ENABLED|DB_USERNAME|OPTIONAL_TUNING|CFG_)" | sort'
```

Expected output:

```
CFG_APP_GREETING=Meridian Freight booking API
CFG_FEATURE_BULK_UPLOAD=true
CFG_FEATURE_LEGACY EDI=false
CFG_FEATURE_LIVE_VESSEL ETA=false
CFG_FEATURE_MULTILEG_QUOTES=true
DB_USERNAME=DUMMY-not-a-real-username
GREETING=Meridian Freight booking API
MULTILEG_ENABLED=true
```

Four things to read out of that list:

1. OPTIONAL_TUNING is absent. Its configMapKeyRef set optional: true and the key does not exist, so the variable was simply not created - no error. Compare with section 6, where the same situation without optional kills the container.
2. CFG_app-settings.json is absent. envFrom skips keys that are not valid shell identifiers. kubelet records a warning event instead of failing:

```
kubectl get events -n kcna-lab15 --field-selector involvedObject.name=env-consumer
```

Expected output (an InvalidEnvironmentVariableNames warning; wording may vary slightly by release):

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
39s	Normal	Scheduled	pod/env-consumer	Successfully assigned kcna-lab15/env-consumer to kind-control-plane
38s	Warning	InvalidEnvironmentVariableNames	pod/env-consumer	Keys [app-settings.json] from the EnvFrom configMap kcna-lab15/booking-app-config were skipped since they are considered invalid environment variable names.
38s	Normal	Pulled	pod/env-consumer	Container image "busybox:1.36" already present on machine
38s	Normal	Created	pod/env-consumer	Created container: shell
38s	Normal	Started	pod/env-consumer	Started container shell

3. prefix: CFG_ namespaced the bulk import so it cannot collide with the application's own variables.

4. DB_USERNAME holds the decoded secret value. By the time it reaches the process it is plaintext - there is no such thing as an "encrypted environment variable".

Now see how leaky an environment variable is:

```
kubectl exec -n kcna-lab15 env-consumer -- sh -c 'tr "\0" "\n" < /proc/1/environ | grep DB_USERNAME'
```

Expected output:

```
DB_USERNAME=DUMMY-not-a-real-username
```

Anything that can read /proc inside that container - a sidecar sharing the process namespace, a debugging tool, a crash handler writing a core dump - can read every credential you injected as an environment variable.

Step 8 - Inject as volumes, and meet the subPath trap

```
kubectl apply -f manifests/40-pod-volume-injection.yaml
kubectl wait --for=condition=Ready pod/volume-consumer -n kcna-lab15 --timeout=90s
```

Expected output:

```
pod/volume-consumer created
pod/volume-consumer condition met
```

```
kubectl exec -n kcna-lab15 volume-consumer -- ls -l /etc/booking/config /etc/booking/credentials
```

Expected output (..data symlinks again - the atomic-update mechanism):

```
/etc/booking/config:
total 0
lrwxrwxrwx    1 root    root          25 Sep  5 10:40 app-settings.json -> ..data/app-
settings.json
lrwxrwxrwx    1 root    root          19 Sep  5 10:40 greeting.txt -> ..data/greeting.txt

/etc/booking/credentials:
total 0
lrwxrwxrwx    1 root    root          24 Sep  5 10:40 BOOKING_API_TOKEN -> ..data/
BOOKING_API_TOKEN
lrwxrwxrwx    1 root    root          26 Sep  5 10:40 BOOKING_DB_PASSWORD -> ..data/
BOOKING_DB_PASSWORD
lrwxrwxrwx    1 root    root          26 Sep  5 10:40 BOOKING_DB_USERNAME -> ..data/
BOOKING_DB_USERNAME
```

Note that items: projected only two of the ConfigMap's seven keys, and renamed APP_GREETING to greeting.txt. Without items, every key becomes a file.

Check the file permissions the fsGroup made readable:

```
kubectl exec -n kcna-lab15 volume-consumer -- sh -c 'ls -lL /etc/booking/credentials/BOOKING_DB_PASSWORD; id'
```

Expected output:

```
-r--r----- 1 root      1000          35 Sep  5 10:40 /etc/booking/credentials/
BOOKING_DB_PASSWORD
uid=1000 gid=1000 groups=1000
```

Mode 0440, owner root, group 1000 - set by fsGroup: 1000. Without fsGroup, that file would be root:root and this non-root container could not read it. That is one of the most common "I hardened my Pod and it broke" incidents.

Read the mounted secret:

```
kubectl exec -n kcna-lab15 volume-consumer -- cat /etc/booking/credentials/BOOKING_DB_PASSWORD
; echo
```

Expected output:

```
DUMMY-not-a-real-password-change-me
```

Confirm the Secret volume is tmpfs - never written to the node's disk:

```
kubectl exec -n kcna-lab15 volume-consumer -- sh -c 'mount | grep booking/credentials'
```

Expected output (the source device name varies):

```
tmpfs on /etc/booking/credentials type tmpfs (ro,relatime,size=...)
```

The subPath demonstration

Both mounts below come from the same ConfigMap. They are currently identical:

```
kubectl exec -n kcna-lab15 volume-consumer -- sh -c 'grep log_level /etc/booking/config/app-
settings.json /etc/booking/pinned-settings.json'
```

Expected output:

```
/etc/booking/config/app-settings.json:  "log_level": "info",
/etc/booking/pinned-settings.json:  "log_level": "info",
```

Change the ConfigMap:

```
kubectl patch configmap booking-app-config -n kcna-lab15 --type=merge \
-p '{"data":{"app-settings.json":{"\n  \"service\": \"booking-api\", \n  \"log_level\": \
\"debug\"\n}\n}}'
```

Expected output:

```
configmap/booking-app-config patched
```

Now wait for kubelet's sync. This takes up to about 90 seconds - that is normal, not a hang:

```
for i in $(seq 1 20); do
  if kubectl exec -n kcna-lab15 volume-consumer -- grep -q debug /etc/booking/config/app-
settings.json 2>/dev/null; then
    echo "directory mount updated after ~$((i * 5))s"; break
  fi
  sleep 5
done
```

Expected output (the exact number of seconds varies with the kubelet sync period):

```
directory mount updated after ~65s
```

Now compare the two mounts again:

```
kubectl exec -n kcna-lab15 volume-consumer -- sh -c 'grep log_level /etc/booking/config/app-
settings.json /etc/booking/pinned-settings.json'
```

Expected output:

```
/etc/booking/config/app-settings.json: "log_level": "debug",
/etc/booking/pinned-settings.json: "log_level": "info",
```

There is the trap. The directory mount updated. The subPath mount did not, and never will for the life of the container.

Why: a whole-directory ConfigMap mount is managed by kubelet as an atomically swapped symlink tree, so refreshing it is a symlink flip. A subPath mount is a bind mount of one file made at container-creation time; kubelet has no way to re-point it without restarting the container.

****And note what did *not* happen anywhere: no environment variable changed. env-consumer still reports the old values, because environment variables are frozen at process start. If your application must pick up config changes without a restart, it must read a whole-directory volume mount** and re-read the file.**

Restore the ConfigMap before verification:

```
kubectl apply -f manifests/10-configmap-booking-app-config.yaml
```

Expected output:

```
configmap/booking-app-config configured
```

5. Verification

```
bash verification/checks.sh
```

Expected output:

```

== Lab 15 verification: Secrets, ConfigMaps and safe injection ==
PASS namespace kcna-lab15 exists
PASS ConfigMap booking-app-config app-settings.json matches data/app-settings.json byte-for-byte
PASS ConfigMap booking-app-config has all 4 keys from data/feature-flags.env with matching values
PASS ConfigMap booking-app-config-v1 is immutable
PASS Secret booking-credentials has all 3 keys from data/dummy-credentials.env
PASS Secret booking-credentials values base64-decode to the data/dummy-credentials.env values
PASS Secret booking-credentials contains only obvious DUMMY placeholders
PASS Secret booking-api-example stores stringData as base64 under .data and returns no stringData
PASS env-consumer resolved GREETING from a configMapKeyRef
PASS env-consumer resolved DB_USERNAME from a secretKeyRef
PASS env-consumer bulk-imported the FEATURE_* keys with the CFG_ prefix
PASS env-consumer omitted the optional key that does not exist
PASS volume-consumer projected only the 2 requested ConfigMap keys, with greeting.txt renamed
PASS volume-consumer secret file mode is 0440 with group 1000 (fsGroup)
PASS volume-consumer secret volume is tmpfs (never written to node disk)
PASS volume-consumer subPath mount exists alongside the directory mount
PASS RECORDED: this cluster has NO --encryption-provider-config (Secrets are plaintext in etcd)
-- 17 passed, 0 failed --

```

6. Failure injection - a key name that does not exist

6.1 Break it

```
kubectl apply -f manifests/50-pod-broken-key.yaml
```

Expected output - accepted, because nothing validates key names at admission:

```
pod/broken-key-consumer created
```

```
sleep 10; kubectl get pod broken-key-consumer -n kcna-lab15
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
broken-key-consumer	0/1	CreateContainerConfigError	0	12s

CreateContainerConfigError is specific and diagnostic: the image pulled fine, the Pod was scheduled fine, and kubelet then failed while assembling the container's configuration.

6.2 Get the real error text

```
kubectl describe pod broken-key-consumer -n kcna-lab15 | tail -14
```

Expected output:

```

Events:
  Type      Reason      Age           From              Message
  ----      -
  Normal    Scheduled   25s           default-scheduler Successfully assigned kcna-lab15/
broken-key-consumer to kind-control-plane
  Normal    Pulled      10s (x3 over 24s) kubelet           Container image "busybox:1.36"
already present on machine
  Warning   Failed      10s (x3 over 24s) kubelet           Error: couldn't find key APP_TIER
in ConfigMap kcna-lab15/booking-app-config

```

The one line that matters:

```
Error: couldn't find key APP_TIER in ConfigMap kcna-lab15/booking-app-config
```

It names the key, the namespace and the object. The same shape appears for a missing Secret key (couldn't find key ... in Secret ...) and, if the object itself is absent, configmap "..." not found.

6.3 Diagnose

List the keys that actually exist:

```
kubectl get configmap booking-app-config -n kcna-lab15 -o jsonpath='{range .data}{" "}{end}'
kubectl get configmap booking-app-config -n kcna-lab15 -o go-template='{range $k, $v := .
data}}{{ $k }}{"\n"}}{{end}}' | sort
```

Expected output:

```
APP_GREETING
FEATURE_BULK_UPLOAD
FEATURE_LEGACY_EDI
FEATURE_LIVE_VESSEL_ETA
FEATURE_MULTILEG_QUOTES
app-settings.json
```

There is no APP_TIER. Compare with what the Pod asked for:

```
kubectl get pod broken-key-consumer -n kcna-lab15 \
-o jsonpath='{range .spec.containers[0].env[*]}{.name}{" <- "}{.valueFrom.configMapKeyRef.
name}{"\n"}}{{end}}'
```

Expected output:

```
APP_TIER <- booking-app-config/APP_TIER
```

Diagnosis: the reference is to a key that was never defined. Nothing catches this until kubelet tries to build the container - not kubectl apply, not admission, not kubeconform:

```
kubeconform -strict -summary manifests/50-pod-broken-key.yaml
```

Expected output:

```
Summary: 1 resource found parsing 1 file - Valid: 1, Invalid: 0, Errors: 0, Skipped: 0
```

6.4 Fix - two valid choices

Choice A - add the key (the config really was missing):

```
kubectl patch configmap booking-app-config -n kcna-lab15 \
  --type=merge -p '{"data":{"APP_TIER":"training"}}'
kubectl delete pod broken-key-consumer -n kcna-lab15
kubectl apply -f manifests/50-pod-broken-key.yaml
kubectl wait --for=condition=Ready pod/broken-key-consumer -n kcna-lab15 --timeout=90s
kubectl exec -n kcna-lab15 broken-key-consumer -- printenv APP_TIER
```

Expected output:

```
configmap/booking-app-config patched
pod "broken-key-consumer" deleted
pod/broken-key-consumer created
pod/broken-key-consumer condition met
training
```

The Pod had to be recreated. A Pod's env is immutable; patching the ConfigMap

alone would have left it stuck in CreateContainerConfigError forever.

Choice B - mark the reference optional (the config is genuinely not always present): add optional: true under the configMapKeyRef, exactly as OPTIONAL_TUNING does in manifests/30-pod-env-injection.yaml. The container then starts with the variable simply unset.

Restore the ConfigMap and remove the failure-injection Pod before cleanup:

```
kubectl delete pod broken-key-consumer -n kcna-lab15
kubectl apply -f manifests/10-configmap-booking-app-config.yaml
```

Expected output:

```
pod "broken-key-consumer" deleted
configmap/booking-app-config configured
```

7. Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix	
CreateContainerConfig Error	configMapKeyRef/secretKeyRef names a key that does not exist	kubectl describe pod <p> → couldn't find key <K> in ConfigMap <ns>/<name>	Add the key, or set optional: true, then recreate the Pod	
CreateContainerConfig Error with configmap "x" not found	The whole object is missing, or is in another namespace	kubectl get cm -n <ns>	Create it in the same namespace - ConfigMaps and Secrets never cross namespaces	
Field is immutable on patch	The ConfigMap/Secret has immutable: true	kubectl get cm <n> -o jsonpath='{.immutable}'	Create a new versioned object (...-v2) and roll the workload	
A ConfigMap change is not visible in the container	Injected as env (frozen at start), or mounted with subPath	`kubectl get pod <p> -o yaml \	grep -A2 subPath`	Use a whole-directory volume mount and re-read the file, or roll the Deployment
Config change visible in the container but the app ignores it	The application caches the file at startup	Read the app's own logs	Roll the Deployment; volume updates do not restart processes	
Permission denied reading a mounted Secret as a non-root user	Files are root:root; defaultMode excludes other	kubectl exec <p> -- ls -l <path>; id	Set securityContext.fsGroup to the container's GID, or widen defaultMode	
Key from envFrom never appears	Not a valid shell identifier (contains ., -, or starts with a digit)	kubectl get events → InvalidEnvironmentVariableNames	Use an explicit configMapKeyRef with a valid name:, or a volume mount	
Secret value looks like gibberish in -o yaml	It is base64, not encryption	`... -o jsonpath='{.data.<K>}' \	base64 -d`	Nothing to fix - but stop treating base64 as protection
base64: invalid option -- 'd'	Older macOS base64	base64 --help	Use base64 -D	

8. Cleanup

Delete only this lab's namespace. ConfigMaps, Secrets and Pods are all namespaced.

```
kubectl delete namespace kcna-lab15
```

Expected output:

```
namespace "kcna-lab15" deleted
```

Confirm:

```
kubectl get namespace kcna-lab15
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab15" not found
```

This lab creates no cluster-scoped objects. Never run

```
kubectl delete secret --all without -n kcna-lab15.
```

9. What you learned

- `--from-literal`, `--from-file` and `--from-env-file` produce **different key layouts** from the same data: one key, one key holding a whole file, or one key per line.
- Environment injection is simple but frozen at start, leaks through `/proc/<pid>/environ`, and hard-fails on a missing key unless optional: `true`.
- Volume injection updates live (whole-directory mounts), keeps values out of the process environment, and is the right default for credentials.
- `subPath` mounts are point-in-time copies and never update. The directory mount next to it does.
- `fsGroup` is what makes a 0440 Secret file readable by a non-root container.
- Secret volumes are tmpfs - they never touch the node's disk. data in etcd, however, is plaintext by default.
- A Secret is base64-encoded, not encrypted. `kubectl get -o jsonpath | base64 -d` is a one-liner. The real controls are RBAC, encryption at rest (`--encryption-provider-config`, ideally a KMS v2 provider) and an external secret store.
- describe secret masks values; describe configmap does not. Neither is a security boundary.
- `immutable: true` protects running Pods and reduces API-server load, at the cost of forcing you to version the object's name.
- Missing-key failures are invisible to `kubectl apply` and to static validation. They surface only as `CreateContainerConfigError` at container start.

10. Further reading

- ConfigMaps - <https://kubernetes.io/docs/concepts/configuration/configmap/>
- Secrets - <https://kubernetes.io/docs/concepts/configuration/secret/>

- Configure a Pod to Use a ConfigMap -
<https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/>
- Distribute Credentials Securely Using Secrets -
<https://kubernetes.io/docs/tasks/inject-data-application/distribute-credentials-secure/>
- Encrypting Confidential Data at Rest - <https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>
- Using a KMS provider for data encryption - <https://kubernetes.io/docs/tasks/administer-cluster/kms-provider/>
- Good practices for Kubernetes Secrets - <https://kubernetes.io/docs/concepts/security/secrets-good-practices/>
- Configure a Security Context (fsGroup) -
<https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>